

OOP

programare orientată pe obiecte

CURS05: ABSTRACTIZARE

INTERFETE SI CLASE ABSTRACTE

Alexandru Olteanu

{alexandru.olteanu@upb.ro}

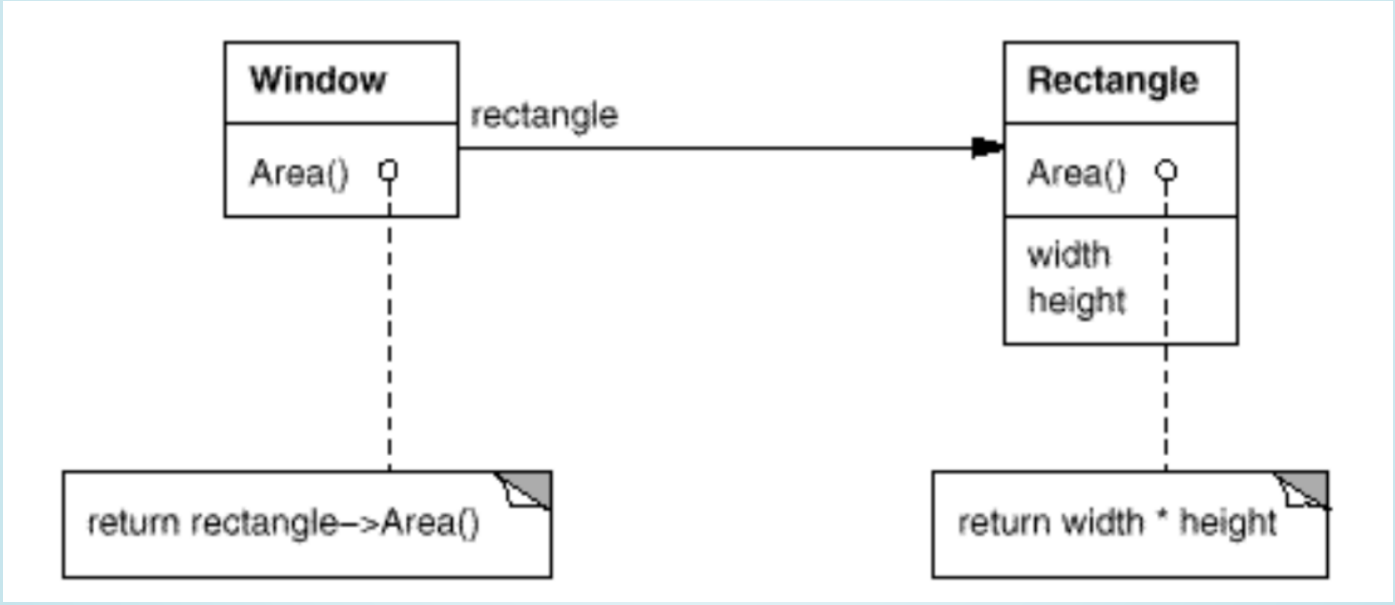
Computer Science & Engineering Department

FACULTATEA DE AUTOMATICĂ ŞI CALCULATOARE

Universitatea Politehnica Bucureşti

DELEGATION

Pentru a putea schimba implementarea la run-time, avem nevoie de interfete



Delegation, Introduction, "Design Patterns", Gang of Four

INTERFETE

alexandru.olteanu@upb.ro

CE ESTE O INTERFATA?





alexandru.olteanu@upb.ro

Exemplu definire interfata

```
public interface Payee {
    int computeMoney ( Month paymentMonth ) ;
}
public class Engineer implements Payee {
    private Contract contract ;
    public int computeMoney ( Month paymentMonth ) {
        if ( daysWorked + paidLeave == paymentMonth.getWorki
            return contract.getSalary () ;
    }
}
public class SalesRep implements Payee {
    private Contract contract ;
    private int newCustomers ;
    public int computeMoney ( Month paymentMonth ) {
        return contract.getSalary () + newCustomers * contra
    }
```

Exemplu folosire interfata

```
public class FinancialManager {
    private BankAccount ba;
    private List <Payee > payroll ;
    public executePayroll ( Month paymentMonth ) {
        for ( Payee payee : payroll ) {
            ba.pay( payee.computeMoney ( paymentMonth ) ) ;
        }
    }
}
```

Din punct de vedere payroll, aplicatia va functiona indeferent cati si ce fel de angajati are firma.

Interfata este un tip de date care reprezinta o grupare de semnaturi de metode (fara implementari) unite de un scop comun.

Interfata este un contract care defineste interactiunea a doua componente software. Permite:

- Schimbarea dinamica a implementarilor
- Decuplare pentru reutilizare si extindere

e.g. pentru spital, FinancialManager ramane la fel

```
public interface Payee {
    int computeMoney ( Month paymentMonth ) ;
}
public class Nurse implements Payee {
    private Contract contract ;
    public int computeMoney ( Month paymentMonth ) {
        if ( daysWorked + paidLeave == paymentMonth .
                getWorkingDays () )
            return contract . getSalary () ;
    }
}
public class Doctor implements Payee {
    private Contract contract ;
    public int computeMoney ( Month paymentMonth ) {
        return contract . getSalary () + guardShifts * contr
                getGuardShiftBonus () ;
```

Ce poate contine o interfata?

Pana la Java 8:

- semnaturi de metode
- metode statice (inclusiv implementari)
- constante (static final)

```
public interface Payee {
        static final String PAYER = " SuperStar Basket Club "; /
        static final Bank PAYING_BANK = " Rich Swiss Bank "     /
        static String printPayerData () {
            System .out. println ( PAYING_BANK +" on the behalf
                    PAYER ) ;
        }
}
```

Toate sunt public in mod automat (nu se trec specificatori de acces)

Ce poate contine o interfata?

De la Java 8:

- semnaturi de metode
- metode statice (inclusiv implementari)
- constante (static final)
- metode default (inclusiv implementari) cu implicatii asupra mostenirii multiple

```
public interface Payee {
  default Contract signContract () {
      return new Contract (" Payer Inc.") ;
  }
}
```

Toate sunt public in mod automat (nu se trec specificatori de acces)

MOSTENIREA MULTIPLA

alexandru.olteanu@upb.ro

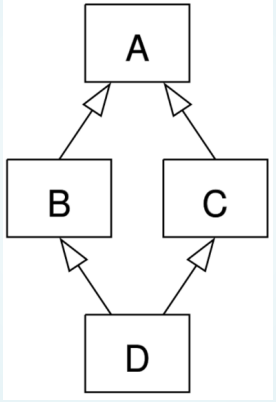
Mostenirea multipla este o functionalitate oferita de unele limbaje de programare prin care o clasa derivata preia caracteristicile mai multor clase parinte.

Mostenire multipla vs limbaje de programare

- mostenirea multipla in sensul clasic (oferita de limbaje precum C++) in care o clasa poate mostenii mai multe clase
- in Java < 8: o clasa poate extinde o singura alta clasa, o clasa poate implementa mai multe interfete
- in Java 8+ : o interfata poate contine metode default, o clasa poate sa implementeze mai multe interfete cu metode default

PROBLEMA DIAMANT

Problema diamant ("diamond problem", aka "deadly diamond of death") se refera la ambiguitatea care apare atunci cand doua clase B si C mostenesc o clasa A, iar o alta clasa D mosteneste atat B, cat si C.



Daca exista o metoda in A, pe care o suprascriu atat B, cat si C, dar D nu o suprascrie, atunci D pe care din cele doua variante o va mosteni?

Programatorului i se cere sa rezolve ambiguitatea

TIPURI DE MOSTENIRE MULTIPLA

- Multiple Inheritance of Type: suportat de Java, vezi Delegation
- Multiple Inheritance of State: nu e suportat de Java, interfetele nu au campuri
- Multiple Inheritance of Implementation: nu e suportat Java < 8, devine suportat din Java 8+ din metode default

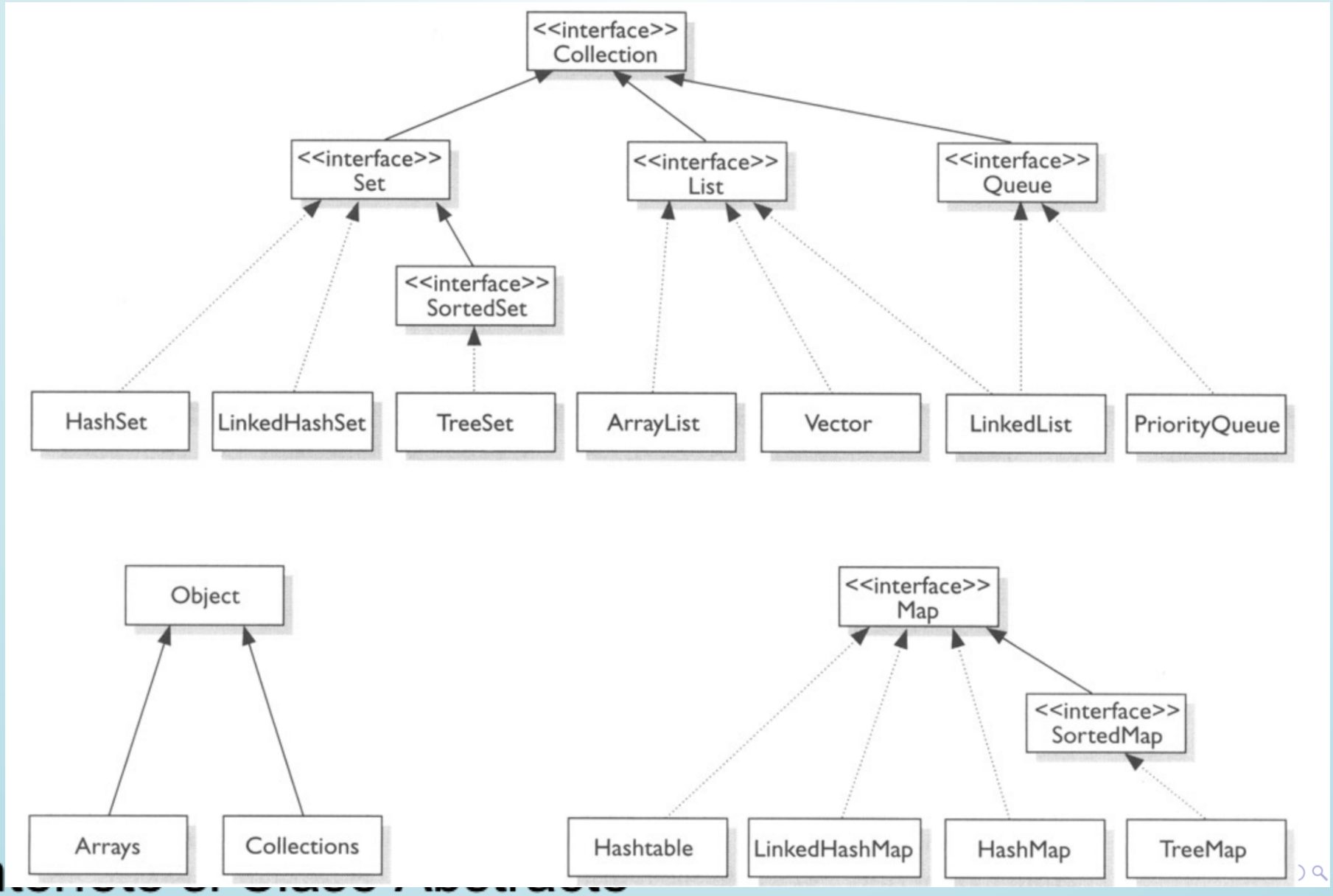
Multiple inheritance in Java

Interfete celebre

```
List <CEO> topCEOs = new ArrayList <CEO>() ;
topCEOs.add(new CEO (" Steve Jobs ") ) ;
topCEOs.add(new CEO (" Bill Gates ") ) ;
topCEOs.add(new CEO (" Jeff Bezos ") ) ;
topCEOs.add(new CEO (" Elon Musk ") ) ;
```

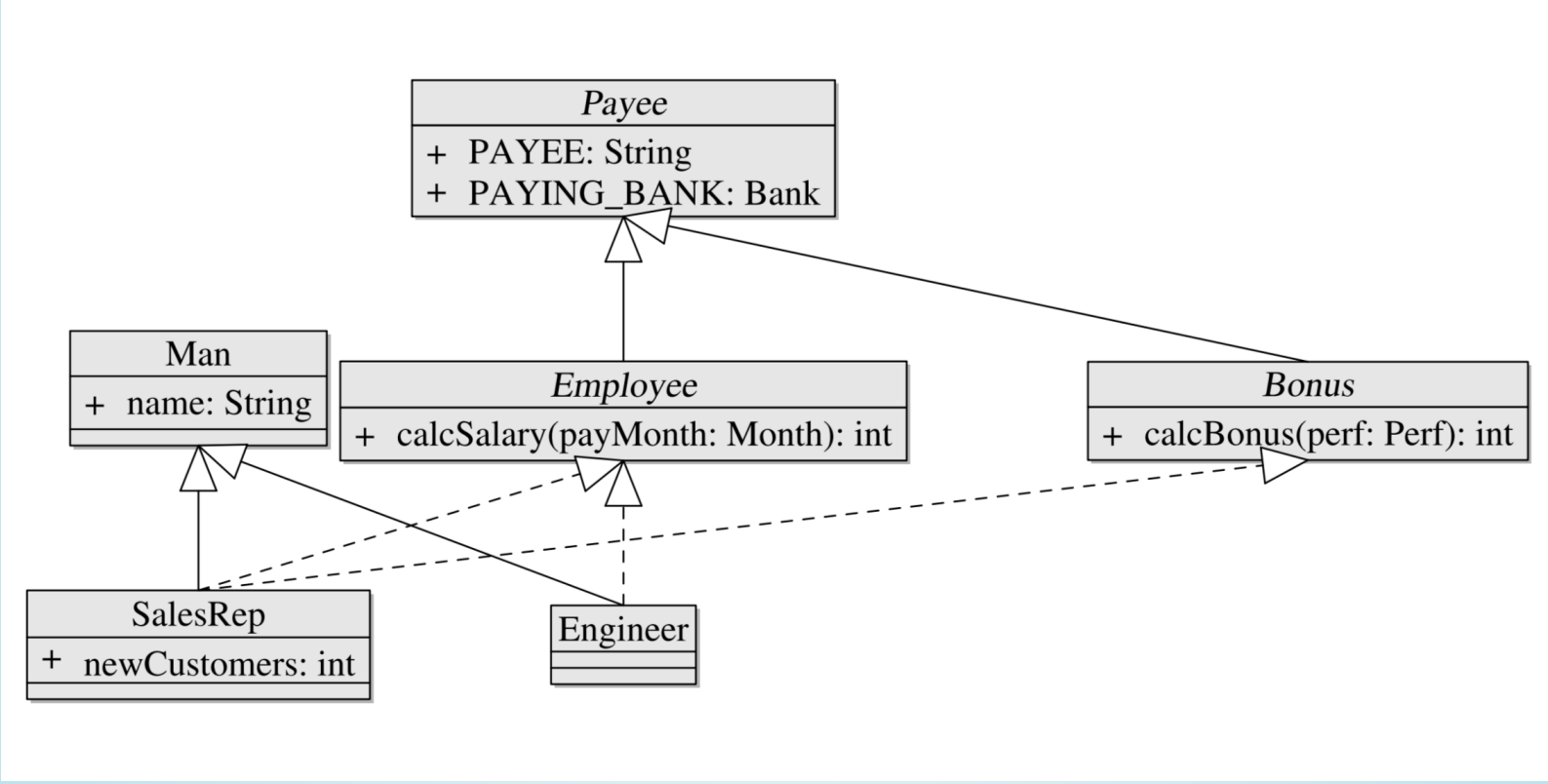
```
public class CEO implements Comparable {
  public int netWorth ;
  public int compareTo ( CEO otherCEO ) {
    return netWorth - otherCEO.netWorth () ;
  }
}
. . .
Collections . sort ( topCEOs ) ;
```

Mostenirea interfetelor


<<interface>>
Collection
<<interface>>
Set
<<interface>>
List
<<interface>>
Queue
<<interface>>
SortedSet
HashSet
LinkedHashSet
TreeSet
ArrayList
Vector
LinkedList
PriorityQueue
Object
Arrays
Collections
<<interface>>
Map
<<interface>>
SortedMap
Hashtable
LinkedHashMap
HashMap
TreeMap

: In ale

Mostenirea interfetelor



RELATII INTRE INTERFETE SI INTRE INTERFETE SI CLASE

- o interfata poate extinde mai multe interfete (mostenire)
- o interfata poate implementata de mai multe clase
- o interfata nu poate fi instantiata
- o clasa poate implementa mai multe interfete

CLASE ABSTRACTE

alexandru.olteanu@upb.ro

CLASELE ABSTRACTE CA ENTITATI INTERMEDIARE INTRE CLASE SI INTERFETE

e.g. Payee:

- signContract este valabil pentru toti cei care sunt platiti de companie, deci poate implementat din start
- calcMoney este variabil in functie de cei care extind Payee (Employee, Bonus) deci poate declarat abstract si lasat spre implementare acestora

Clasa abstracta este o clasa declarata ca abstracta prin utilizarea cuvantului cheie "abstract". Desi nu este sintactic obligatoriu, acest lucru are sens sa se intample daca cel putin una din metodele sale este abstracta (adica declarata fara implementare).

Reciproca insa este obligatorie: daca o clasa are cel putin o metoda abstracta, toata clasa trebuie marcata ca abstracta.

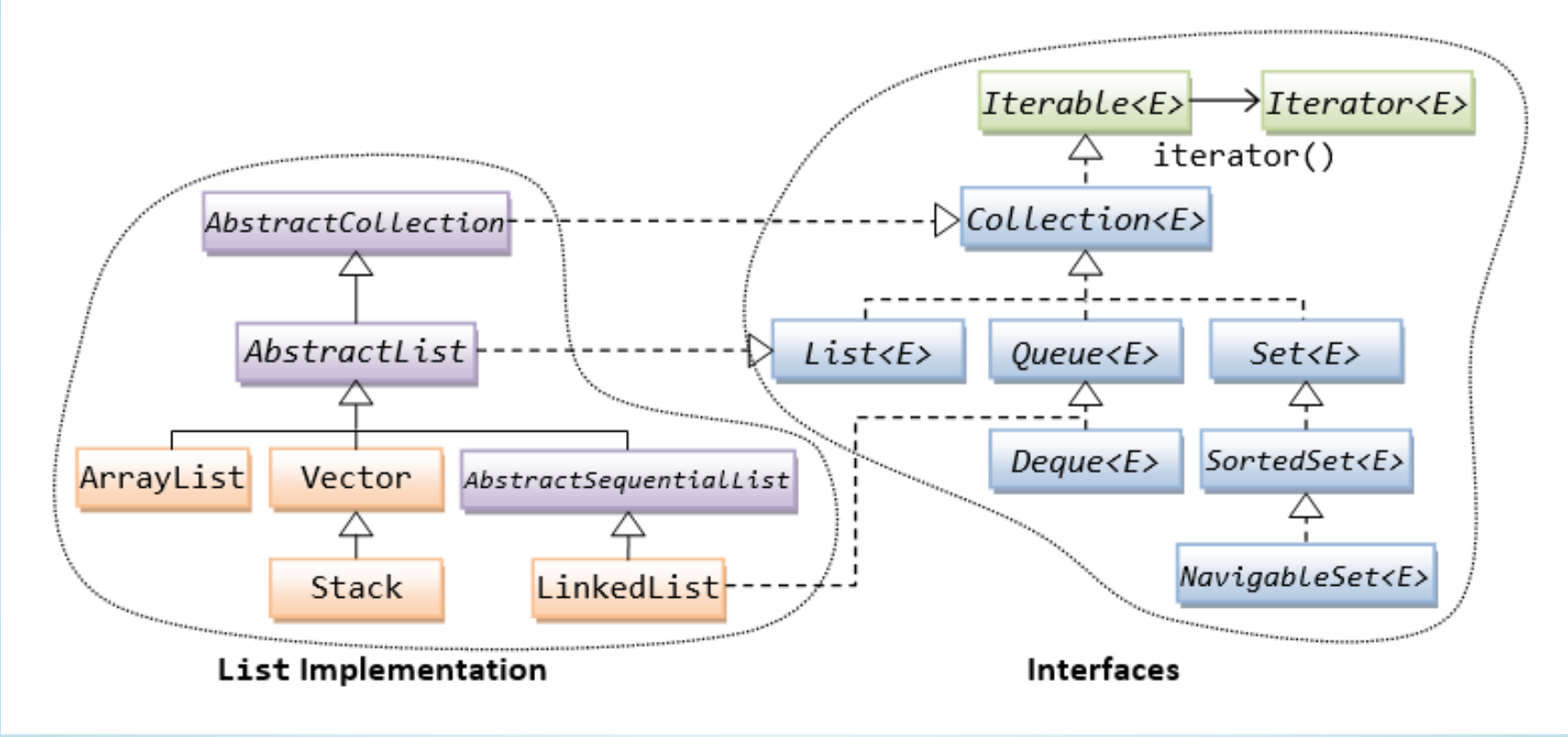
Ce poate contine o clasa abstracta?

- orice contine o clasa obisnuita, doar ca este marcata ca abstracta (de fapt o clasa obisnuita poate marcata ca abstracta fara vreun motiv anume)
- elegant ar fi ca cel putin una din metodele unei clase abstracte sa e abstracta (adica semnatura sa aiba cuvantul cheie abstract si sa nu aiba implementare)
- specificatorii de acces se pastreaza ca la o clasa obisnuita.

Clasa abstracta ofera o cale de mijloc intre clase si interfete, oferind posibilitatea de a crea clase incomplete, care vin cu un contract pentru a complet implementate de clasele ce le mostenesc:

- cand denfiim intr-un mod corect ceea ce au in comun clasele derivate
- cand dorim sa oferim o implementare partiala unei clase, ca sa nu duplicam cod (desi din Java 8 acest lucru se poate face si cu interfete si metode default)

Clase abstracte celebre



Ar trebui sa intelegeti tot ce scrie aici:

Why List interface?

Relatii intre clase abstracte, interfete si clase

- o clasa poate extinde o clasa abstracta daca ii implementeaza toate metodele abstracte
- daca o clasa extinde o clasa abstracta si nu ii implementeaza toate metodele abstracte, atunci, la randul sau, va marcata ca abstracta

Interfete vs Clase Abstracte

	Interfata	Clasa Abstracta
Se poate instantia	nu	nu
Metode	cu sau fara implementare	cu sau fara implementare
Campuri	static final (automat)	*de orice fel*
Specificatori acces	public (automat)	*public, private, protected, default*
Relatia cu alte clase	*o clasa poate implementa oricate interfete*	o clasa poate extinde o singura clasa abstracta

Use abstract classes when:

- sharing code among several closely related classes, or
- classes that extend the abstract class have many common methods or fields, or require access modiers other than public, or
- non-static or non-final fields are needed (methods that can access and modify the state of the object to which they belong)

Use interfaces when:

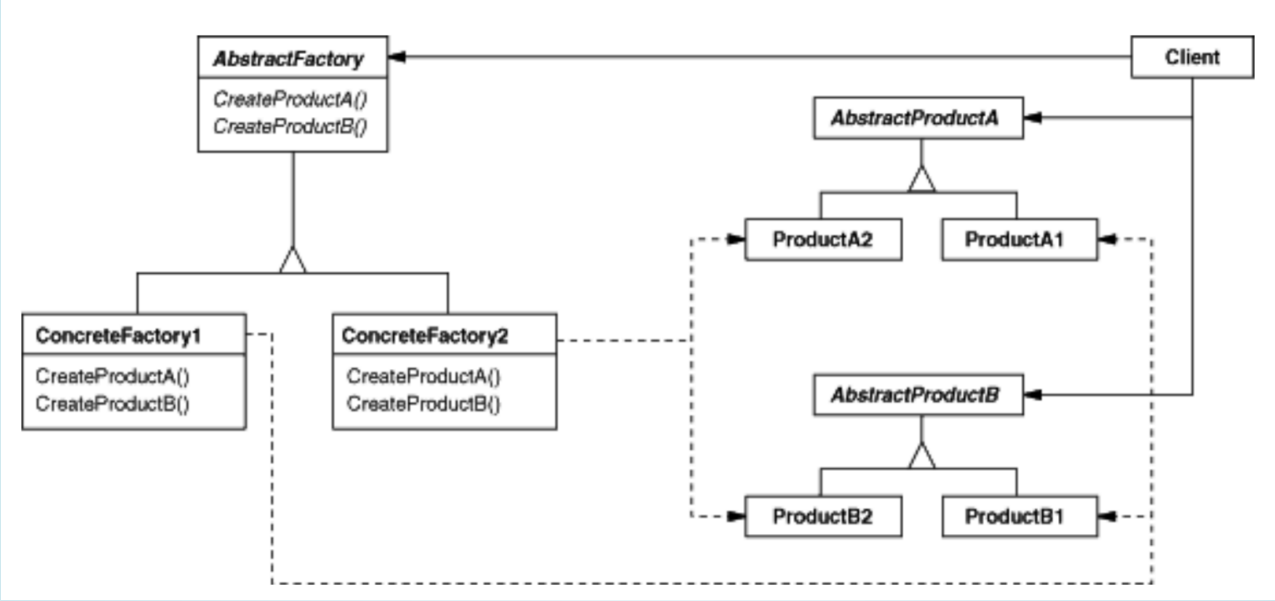
- unrelated classes would implement the interface, or
- specifying the behavior of a particular data type, but not concerned about who implements its behavior, or
- multiple inheritance of type is needed

Oracle tutorial on Abstract Methods and Classes

FACTORY

alexandru.olteanu@upb.ro

Vrem sa generam instante dintr-o ierarhie de clase, fara sa stim toate implementarile posibile, doar caracteristicile obiectului ce trebuie implementat



-- Gang of Four, Design Patterns

- Products like soups, salads, main dishes etc.
- Factories like ItalianRestaurant, AsianRestaurant etc.

EXERCITII

- Exemplu de Factory in C#
- Cautati care sunt clasele Factory in acest Java API
- Ar trebui sa intelegeti tot ce scrie in articol despre factory patterns
- Why I hate frameworks (aka Factory Factory Factory,)